

Using AI Agents to Analyze NFL Data

An LLM-Powered Analytics Agent with SQL, RAG, and HITL Workflows

Jacob Poore & Arian Zarazua

Professor Jennifer Rose

QAC 387: Generative AI — LLM Tools for Data Analysis

March 14, 2026

Abstract

This paper presents the design and development of an AI-powered agent for analyzing NFL statistical data spanning 2010–2025. The agent ingests CSV files and rebuilds a local SQLite database on each initialization rather than relying on a persistent production data layer. The system integrates a natural language to SQL pipeline powered by an OpenAI LLM, LangChain for tool orchestration, and custom Python analytics modules for regression, variance analysis, and visualization. A Streamlit GUI surfaces results to end users, enabling analysts and researchers to query complex multi-season datasets without requiring Python or SQL expertise.

GitHub Repository: <https://github.com/Arian-Zarazua/Jacob-Arian>

Introduction

The vision of this project is an AI agent capable of performing analytics across multiple NFL datasets simultaneously, covering the 2010 through 2025 seasons. The agent enables users to ask questions such as “*What was the average number of passing and rushing yards per game per team in a given year?*” entirely in natural language, without manual querying.

The system lowers the barrier to entry for sports analytics by combining large language model reasoning with structured database querying, retrieval-augmented generation (RAG), and a human-in-the-loop (HITL) approval workflow.

Motivation & Background

Traditional NFL statistical analysis requires domain expertise in SQL or access to expensive proprietary platforms. Our primary datasets are drawn from Pro Football Reference and Stat-Savant, covering team and player statistics across all 30 franchises from 2010 to 2025. A single season’s play-by-play file may contain hundreds of columns and hundreds of thousands of rows, and cross-seasonal comparisons require joining tables with frequently differing schemas.

Our agent addresses three core limitations:

- **Accessibility:** A natural language interface democratizes access to structured sports data.
- **Scale:** Fifteen seasons of NFL data are difficult to synthesize manually; automated query generation collapses this complexity.
- **Timeliness:** Roster changes make static analyses insufficient, so the pipeline is designed for incremental data loading.

Development Overview

System Architecture

The agent’s full workflow is illustrated in Figure 1. The pipeline begins with SQLite and RAG database initialization, passes user prompts through an LLM-powered router, and dispatches work to either a tool-execution path or a code-generation path, each gated by HITL approval before producing output.

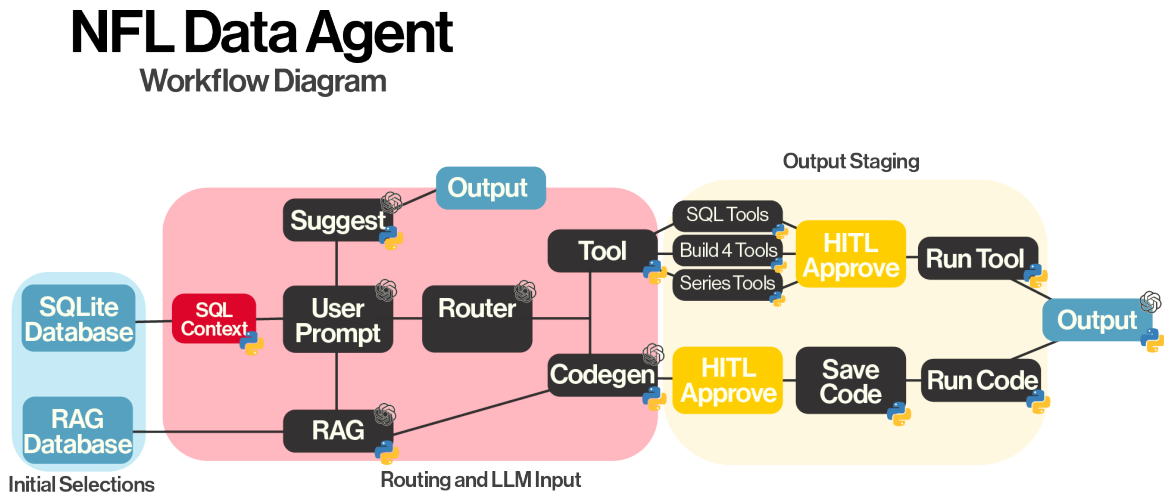


Figure 1: NFL Data Agent workflow: data sources feed the LLM router, which dispatches to HITL-approved tool or code-generation paths.

Technology Stack

Table 1 summarizes the major components and their roles within the system.

Table 1: Technology stack and component roles.

Layer / Tech	Role
OpenAI Model	Core LLM for reasoning, understanding, and response generation
LangChain Routing	Routes user intent, selects tools, and coordinates agent control flow
FAISS / RAG	Retrieves relevant context through vector similarity search
Streamlit	User-facing dashboard for prompt input, outputs, tables, and visualizations
SQLite	Local structured database for queryable data, persistence, and verification
Python Tools	Generates plots, summaries, and analytical outputs from model-selected data
Langfuse	Tracks traces, latency, generations, and overall agent observability

Streamlit Interface

The Streamlit dashboard provides the primary user-facing interface, shown in Figure 2. Users can submit prompts, review generated SQL, inspect dataset schemas, and view analytical visualizations within a unified web application.

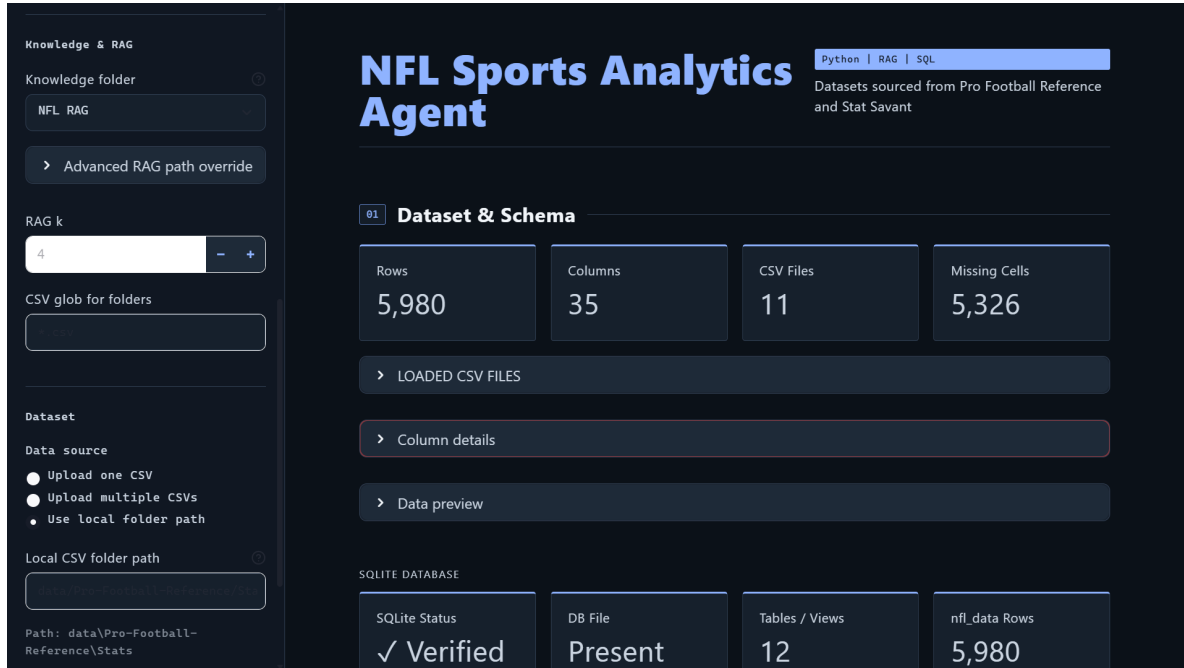


Figure 2: Streamlit dashboard showing dataset and schema overview, including row counts, column counts, loaded CSV files, and SQLite verification status.

Development Process

Data Source & Preprocessing

All data is sourced from Pro Football Reference and Stat-Savant and undergoes the following preprocessing pipeline before being loaded into SQLite:

1. **Loading and directory setup:** Raw CSV files are ingested and output directories initialized.
2. **Missing data handling:** A missingness audit identifies null columns; records with null primary key values are dropped; analytical columns use median imputation within season and position group.
3. **Column classification:** Numeric and categorical features are separated to drive downstream encoding decisions.
4. **Schema normalization:** Year-over-year schema changes are reconciled via a column-mapping configuration file.
5. **Indexing:** Composite indexes are created at load time to minimize query latency.

LLM Query Generation

The query generation module constructs a system prompt containing a condensed schema description, instructions to output only valid SQL, and few-shot examples for common NFL analytics patterns. If execution raises a database error, the error is fed back to the LLM for self-correction in a retry loop (maximum three attempts).

RAG Strategy: Design Pivot

A key design decision was the choice of RAG architecture. The initial approach used a single large FAISS index over all CSVs and documentation. This was replaced with a partitioned SQL + RAG strategy (Figure 3), which provides higher retrieval precision, lower latency, cheaper incremental updates, and better scaling for large sports datasets.

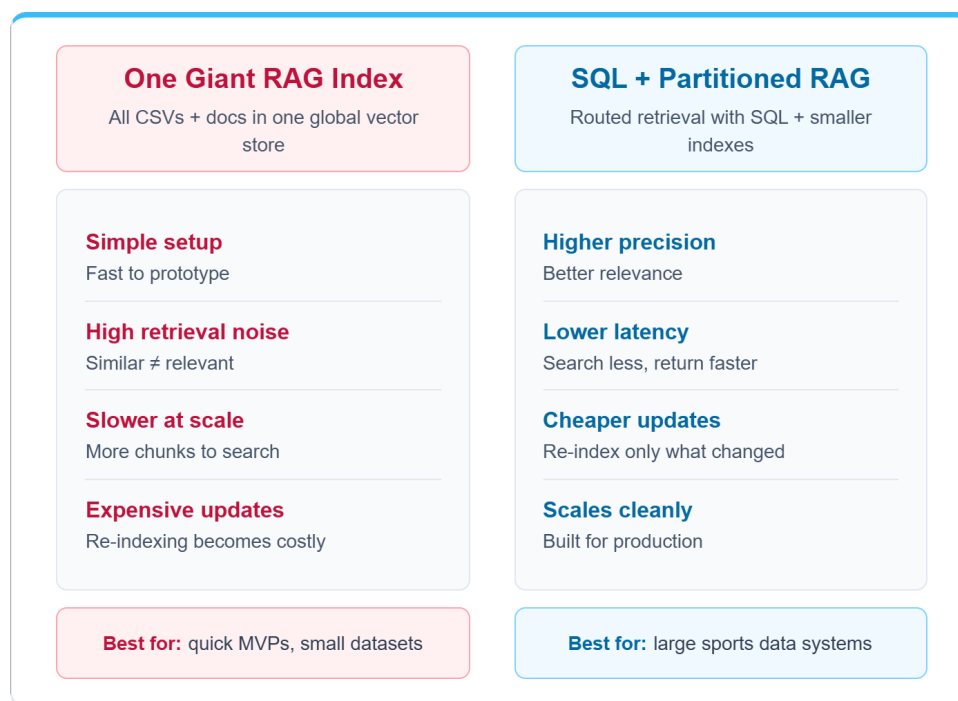


Figure 3: Comparison of RAG strategies: monolithic global index vs. SQL-partitioned retrieval.

Testing & Validation

SQLite Verification

The system performs database health checks using SQLite’s `PRAGMA` commands before every analytical run. `PRAGMA integrity_check` confirms structural validity, while `PRAGMA foreign_key_check` verifies that all table relationships are intact. This acts as a lightweight pre-flight check before any LLM-generated SQL is executed against the database.

Latency Analysis

Table 2 presents Langfuse trace latency percentiles across the four major pipeline stages. Key observations:

- **build-codegen:** p50 of 11.31s and p99 of 17.10s reflects the heavy reasoning required; the narrow 5.7-second spread indicates consistent performance even for complex prompts.
- **build-router:** p99 is more than twice p50, indicating that some prompts require substantially more routing steps.
- **build-run-tool:** p50 of 0.038s confirms that the primary bottleneck is LLM reasoning, not backend execution.

Table 2: Langfuse trace latency percentiles (seconds) across pipeline stages.

Trace Name	p50 (s)	p90 (s)	p95 (s)	p99 (s)
build-codegen	11.306	15.671	16.465	17.101
build-router	4.332	8.703	9.219	9.809
build-summarize	4.781	5.663	5.770	5.856
build-run-tool	0.038	1.608	1.733	1.827

Scoring

Agent outputs were manually evaluated on a 1–10 scale, where 10 represents a fully correct and satisfying result and 1 represents a complete failure to execute. The agent achieved an **average score of 7.563**, reflecting reliable performance across diverse query types with occasional failures on schema-ambiguous or under-specified prompts.

Output Examples

SQL Tools

Table 3 shows output from a natural-language query that the agent routed to the `sql_query` tool, returning average passing yards, rushing yards, and their differential for all 30 NFL markets. Tampa Bay led in average passing yards (275.3yd), while Baltimore showed the smallest pass-rush differential (92.0yd), consistent with their historically run-heavy offensive identity.

Table 3: Agent SQL tool output: average passing yards, rushing yards, and pass-rush differential per market (all seasons). Top 15 of 30 markets shown, ordered by average passing yards descending.

Market	Avg Pass Yds	Avg Rush Yds	Differential
Tampa Bay	275.3	96.7	178.6
New Orleans	274.4	113.1	161.3
Atlanta	270.4	105.6	164.8
Kansas City	270.4	115.4	155.0
Detroit	265.9	102.1	163.8
Pittsburgh	263.2	101.7	161.5
Los Angeles	261.6	104.8	156.8
New England	260.2	114.0	146.2
Green Bay	259.8	116.0	143.8
Dallas	257.1	125.2	131.9
Philadelphia	252.7	128.4	124.3
Cincinnati	250.2	103.5	146.7
Minnesota	249.6	112.2	137.5
Denver	248.6	110.9	137.7
Miami	248.6	100.7	147.8

Code Generation

Figures 4 and 5 show outputs from the agent’s code-generation path. The heatmap reveals a moderate positive correlation between passing yards and passing touchdowns (0.49), and a modest negative correlation between passing and rushing yards (-0.23), consistent with game-level pass-run tradeoffs. The scatter plot with OLS regression line shows a slight negative slope (≈ -0.9 rating points per additional attempt), likely reflecting garbage-time volume inflation in blowout games rather than a causal relationship.

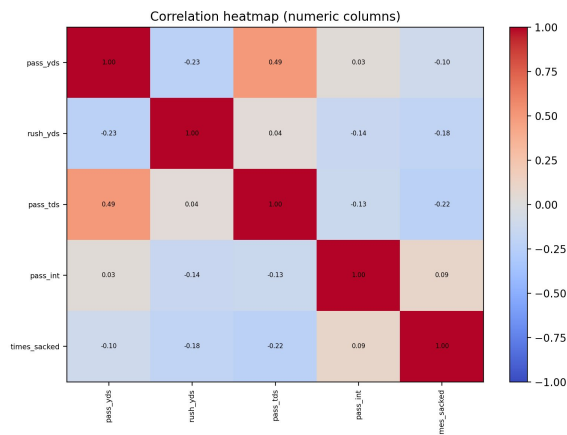


Figure 4: Correlation heatmap of numeric offensive columns.

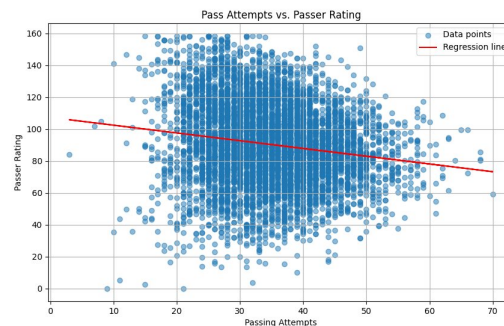


Figure 5: Pass attempts vs. passer rating with OLS regression line.

Challenges & Limitations

Valid-Looking Code, Invalid Analysis

Occasionally the model produces syntactically valid code that yields analytically incorrect results—graphs that appear populated but reflect wrong aggregations, or regression outputs based on mis-specified feature sets. These failures are difficult to detect automatically and motivate the HITL approval step before any generated code is executed.

RAG Retrieval Quality

RAG still requires stronger dataset-specific operational knowledge. The system can misinterpret context, apply incorrect assumptions, or retrieve loosely related information that misleads the prompt. In some cases, retrieved content has actively degraded output quality by introducing irrelevant schema context. Strengthening domain-specific grounding would reduce these errors.

RAG Memory Interference

The current RAG implementation can interfere with the agent’s conversational memory by introducing retrieved context that competes with or overrides active state. Separating persistent memory, task memory, and externally retrieved knowledge is a prerequisite for more reliable multi-turn sessions.

SQLite Scalability

SQLite is well-suited for local single-user execution but is not designed for concurrent multi-user access or production-scale deployment. All agent sessions currently share a single database file, which limits parallelism and introduces locking risk under simultaneous access.

Next Steps

- **Expanded analytical tooling:** The agent already supports correspondence analysis and k-means clustering via code generation. Promoting these to registered tool functions with schema-aware parameters, and exposing tunable controls (number of PCs, number of clusters) in the Streamlit UI, would improve reliability and user control.
- **Improved RAG pipeline:** Separate persistent memory, task memory, and externally retrieved knowledge to prevent context overwriting. Restrict retrieval to domain-specific operational knowledge to improve consistency across complex queries.
- **PostgreSQL migration:** Migrating from SQLite to PostgreSQL would enable reliable multi-user access, better transaction handling, improved performance under concurrent workloads, and scalable cloud deployment with access control.

Summary

This project presents an accessible approach to NFL statistical analysis by combining a natural language interface, LLM-powered SQL generation, LangChain tool orchestration, and a Streamlit deployment. By grounding all analytics in officially sourced datasets and integrating HITL validation throughout the pipeline, the system produces trustworthy, interpretable insights across fifteen seasons of professional football data.

The modular architecture allows new seasons, tools, and analytical models to be integrated with minimal friction. More broadly, this class of AI-powered analytics agents has meaningful potential in data science: it reduces the time and technical overhead required to explore large, complex datasets while still supporting flexible, research-oriented workflows—even for users without backgrounds in SQL or software development.